Global Semiconductor Trade Network

Presented By:

Dani Amin Zeinedeen

Daniel Georges Ghazaly

Charbel Elias Tohme

Presented To:

Dr. Ahmad Abboud

This repository applies Knowledge Representation (KR) techniques to analyze the global semiconductor trade network. By structuring raw customs and trade data into logical frameworks, Semantic Web ontologies, and graph models, this project uncovers hidden supply chain dependencies, pinpoints critical bottlenecks, and enables logic-based risk reasoning and planning.

Part 01: Data Profiling & Cleaning

The first phase establishes a clean, reliable baseline from raw United Nations customs data, narrowing the scope to the year 2024 and resolving structural anomalies before ingestion.

What & Why:

Raw international trade data contains dual perspectives because both importing and exporting nations report the same transactions, often with conflicting numbers. Physical weight metrics are frequently missing or rounded to zero for lightweight, high-value components like semiconductors. This phase standardizes the dataset by dropping unreliable weight metrics, isolating a single year for temporal consistency, and establishing USD trade value as the primary metric.

How it Works (The Mechanics):

- **Manual OpenRefine:** Used initially to filter the dataset strictly to the year 2024, drop all physical weight columns, and remove unresolved geographic aggregates (e.g., _X for "Areas, nes") to prevent the creation of distorted "super-nodes."
- generalProfiling.py: Generates automated HTML data quality reports to visualize missing values and column statistics.
- flippingimport.py: Resolves the UN reporting duality. It identifies all "Import" records, swaps the importer/exporter columns to standardize the flow direction to "Export," and deduplicates the rows to create a single source of truth.
- rule-checks.py: Executes logical assertions to ensure no country trades with itself (irreflexive trades), all trade values are strictly positive, and no orphan nodes exist.

Key Libraries:

- pandas**:** Handles bulk data manipulation, dropping duplicates, and column swapping.
- ydata_profiling**:** Automatically generates comprehensive HTML reports for exploratory data analysis.
- python-dotenv**:** Securely loads environment variables (like local project paths) without hardcoding them into scripts.

Part 02: Data Treatment & Minimization

This phase mathematically aggregates the cleaned dataset, reducing granular, multi-row transactions into a minimized set of distinct network edges and recalculated country profiles.

What & Why:

A knowledge graph operates best when edges represent definitive relationships rather than fragmented ledger entries. By summing all transactions of a specific semiconductor tier between two countries into a single annual edge, the graph becomes leaner and faster to query. Consequently, country-level totals must be recalculated to match the newly minimized edges.

How it Works (The Mechanics):

- transformation_edges.py: Groups the data by exporter, importer, tier, and year. It calculates the sum of all distinct transactions for these groupings, minimizing millions of raw rows into distinct, unified trade flows. Aggregate regional groups are dropped entirely.
- transformation_nodes.py: Rebuilds the country profiles based on the new edges. It computes the true total imports, total exports, and net trade balance for each valid nation to ensure the node-level data perfectly matches the edge-level reality.

Key Libraries:

- pandas**:** Exclusively used here for high-performance grouping, aggregation (groupby, sum), and merging operations directly on the CSV files.

Part 03: RDF Graph & Apache Jena Fuseki

This phase transitions the flat relational data into a queryable semantic web structure using Resource Description Framework (RDF) triples, loaded into an Apache Jena Fuseki triplestore.

What & Why:

Relational databases require complex joins to trace multi-step supply chains. RDF structures data as subject-predicate-object triples, creating a continuous, traverseable graph. We programmatically map relational database rows to RDF triples using R2RML, rather than manually typing INSERT statements, ensuring reproducibility and scalability.

How it Works (The Mechanics):

- load_database.py: Ingests the minimized CSVs (v2) into a PostgreSQL database.
- trade-mapping.ttl: The R2RML recipe that instructs the mapper on how to convert PostgreSQL rows into RDF entities (Trade Observations, Countries, Tiers).
- tier-knowledge.ttl: Explicit background knowledge injected into the graph, primarily linking the 5 semiconductor tiers together via the ul:feeds property to represent the manufacturing sequence.
- materialize.py: Reads the PostgreSQL database and the mapping file to generate semiconductor-knowledge-graph.nt (an N-Triples file containing over 46,000 facts).
- visualize_graph.py: Sorts the graph by USD trade value and extracts only the Top 40 largest transactions to render a localized HTML network diagram, preventing the browser from crashing under the weight of 46,000+ nodes.

Key Libraries:

- morph-kgc**:** A highly efficient engine that executes the R2RML mapping, translating relational SQL data into materialized RDF triples.
- rdflib**:** Used to parse, manipulate, and serialize the resulting graph data.
- pyvis**:** Generates the interactive HTML graph visualizations.
- sqlalchemy / psycopg2**:** Manages the connections and data ingestion for the PostgreSQL database.

Part 04: OWL Ontology Definition & Reasoning

This phase introduces the formal logic layer, defining the structural rulebook of the KRR pipeline.

What & Why:

While RDF provides the graph structure, OWL 2 provides the *meaning*. An ontology is built on top of the graph rather than hardcoding rules into the flat data, allowing inference engines to deduce hidden facts. For example, by defining the feeds property as a TransitiveProperty, the graph automatically understands that if Equipment feeds Wafers, and Wafers feed Processors, then Equipment indirectly feeds Processors.

How it Works (The Mechanics):

- ontology.ttl: The rulebook. It defines core classes (SurplusCountry, DeficitCountry), Functional Properties (each trade has exactly 1 exporter and 1 tier), Transitive Properties (feeds), and Disjointness rules (a Country cannot simultaneously be a Tier, nor can a Surplus Country be a Deficit Country).
- **Protégé & HermiT:** The ontology is loaded into Protégé, where the HermiT reasoner is executed to verify that no logical contradictions exist before it is deployed to the Fuseki triplestore.

Key Libraries / Tools:

- **Apache Jena Fuseki:** The triplestore database hosting the graph and executing complex SPARQL queries.
- **Protégé:** The graphical ontology editor used to design and validate the OWL 2 logic.

Part 05: SHACL Validation & Provenance

An automated quality assurance gate that verifies the materialized data strictly adheres to the ontology's structural requirements before it is utilized by external systems.

What & Why:

A Reasoner (Part 04) infers new knowledge based on logic, but a SHACL gate *validates* existing data against expected shapes. It acts as a firewall. We explicitly separate Violations (which block the pipeline due to structural failures, like missing countries) from Warnings (which flag logical mismatches, like the Kaggle dataset labeling a country a "Net Exporter" when its mathematical trade balance is actually negative).

How it Works (The Mechanics):

- semiconductor-shapes.ttl: Defines the exact structural parameters (e.g., trade values must be a decimal > 0, years must be between 2020 and 2025).
- validate_graph.py: Evaluates the .nt graph against the SHACL shapes. The 2024 graph passes with 0 Violations and 51 Warnings (highlighting original dataset labeling errors).
- record_provenance.py: Generates an immutable PROV-O audit trail (provenance.ttl) utilizing SHA-256 hashes of the data, shapes, and output report to ensure reproducibility.
- shacl-gate.yml: A GitHub Actions CI/CD workflow that blocks repository merges if a single SHACL Violation occurs.

Key Libraries:

- pyshacl**:** The Python engine that executes the SHACL validation rules against the RDF graph locally.
- hashlib**:** Generates the SHA-256 cryptographic hashes for the provenance records.

Part 06: LLM Access Layer (English to SPARQL)

This phase democratizes access to the knowledge graph by translating plain English questions into syntactically valid SPARQL queries using an LLM.

What & Why:

Decision-makers need access to graph insights without learning SPARQL. This pipeline relies on a strict 5-step scaffolding mechanism to prevent the LLM from hallucinating queries or querying non-existent tables.

How it Works (The Mechanics):

1. **Ground:** make_void.py dynamically scans the triplestore to generate void.ttl (schema structure) and vocab.json (exact entity names). The LLM reads this "map" rather than guessing.
2. **Show:** examples.ttl provides few-shot learning by demonstrating 9 exact question-to-SPARQL translations.
3. **Check:** check.py intercepts the LLM's output locally. It ensures the query is a SELECT or ASK statement (blocking INSERT/DELETE injection attacks) and verifies every class and property exists in the VoID map.
4. **Repair:** If check.py finds errors, repair_prompt sends the specific error trace back to the LLM for correction (max 2 attempts).
5. **Refuse:** The LLM is explicitly instructed to refuse questions that fall outside the schema (e.g., pricing forecasts).
6. Evaluate: evaluate.py: Tests the pipeline's scientific accuracy by calculating the **F1 Score**—comparing the exact rows fetched by the LLM's query against a human-written "gold standard" query.

Key Libraries:

- rdflib.plugins.sparql.prepareQuery**:** Used locally to parse the LLM's SPARQL algebra for syntax and security checks before execution.
- urllib **/** json**:** Manages the lightweight HTTPS communication with the Gemini API and the local Fuseki instance.

Part 07: Machine Learning & Link Prediction

The final phase applies graph embedding models to predict hidden, future, or missing supply chain relationships based on the network's established topology.

What & Why:

Knowledge graphs represent known facts, but supply chains contain blind spots. By training a TransE model on the existing triples, the system learns the geometric relationships between entities. It can then predict missing links, such as determining the most probable semiconductor tier traded between two countries with no historical data.

How it Works (The Mechanics):

- build_graph.py: Transforms the CSV edges into a PyTorch Geometric format. It employs specific hiding strategies (split="by_tier", split="by_country", split="random") to mask existing edges, creating a test set to evaluate if the model can accurately predict the missing links.
- link_prediction.py: Trains the TransE model and executes three core query patterns:
    - **Q1:** (Exporter, Importer) → Predict the Tier.
    - **Q2:** (Importer, Tier) → Predict the Exporter.
    - **Q3:** (Exporter, Tier) → Predict the Importer.
- **Evaluation:** The model is evaluated using **MRR** (Mean Reciprocal Rank) and **Hits@10** against a simple Popularity baseline to prove it learned geometric patterns rather than just memorizing the most common trades.
- ui.py: A Flask web server providing a user-friendly interface to test the three query patterns interactively, returning predictions with Softmax confidence percentages.

Key Libraries:

- torch / torch_geometric**:** Provides the deep learning tensors and graph data structures required for the embedding space.
- pykeen**:** A specialized knowledge graph embedding library that handles the TransE pipeline, triple factories, and model evaluation.
- flask**:** Hosts the lightweight local web interface for the predictive queries.